



**SIES GST, NERUL, NAVI MUMBAI  
DEPARTMENT OF COMPUTER ENGINEERING**

**LAB MANUAL**

**MACHINE LEARNING (CSL7001)  
B.E. (COMPUTER ENGINEERING)  
SEMESTER-VII  
(R-2019)**

## COMPUTER ENGINEERING DEPARTMENT

### DEPARTMENT'S VISION

To be a centre of Excellence in Computer Engineering to fulfill the rapidly growing needs of the Society.

### DEPARTMENT'S MISSION

M1: To Impart quality education to meet the professional challenges in the area of Computer Engineering.

M2: To create an environment for research, innovation, professional and social development.

M3: To nurture lifelong learning skills for achieving professional growth.

M4 To strengthen the alumni and industrial interaction for overall development of students.

### PROGRAMME EDUCATIONAL OBJECTIVES(PEOs)

PEO1: Practice Computer engineering in core and multi-disciplinary domains.

PEO2: Exhibit leadership skills for professional growth.

PEO3: Pursue higher Studies for career advancement

### PROGRAMME OUTCOMES (POs)

PO1: Engineering knowledge: Apply the knowledge of mathematics science engineering fundamentals and an mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems engineering problems.

PO2: Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3: Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations

PO4: Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5: Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6: The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7: Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development and need for sustainable development.

PO8: Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9: Individual and team work: Function effectively as an individual and as a member or leader in diverse teams and individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10: Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11: Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12: Life-long learning: long learning: Recognize the need for and have the preparation and ability to engage in independent and lifelong learning in the broadest context of technological change.

## **PROGRAMME SPECIFIC OUTCOMES (PSOs )**

PSO1: To apply computational and logical skills to solve Computer engineering problems.

PSO2: To develop interdisciplinary skills and acquaint with cutting edge technologies in software industries.

### **GENERAL INSTRUCTIONS (Do's And Don'ts)**

1. Wearing ID-Card is compulsory.

2. Keep your bag at the specified place.
3. Shut down the system after use.
4. Place the chairs in proper position before leaving the laboratory.
5. Report failure/Non-working of equipment to Faculty In-charge / Technical Support staff immediately.
6. Know the location of the fire extinguisher and the FIRST-AID Box and how to use then in case of an emergency.
7. Do not eat or drink in the laboratory.
8. Do not litter in the laboratory.
9. Avoid stepping on electrical wires or any other computer cables.
10. Do not open the system unit casing or monitor casing particularly when the power is turned ON.
11. Do not insert metal objects such as clips, pins and needles into the computer casing. They may cause fire.
12. Do not remove anything from laboratory without permissions.
13. Do not touch, connect or disconnect any plug or cable without permission.

### LAB OUTCOMES (LO)

|      |                                                                                |
|------|--------------------------------------------------------------------------------|
| LO 1 | To implement an appropriate machine learning model for the given application.  |
| LO 2 | To implement ensemble techniques to combine predictions from different models. |
| LO 3 | To implement the dimensionality reduction techniques.                          |

### LAB ARTICULATION MATRIX (MAPPING WITH PO & PSO)

|             | Lab Outcome                                                                    | PO 1 | PO 2 | PO 3 | PO 4 | PO 5 | PO 6 | PO 7 | PO 8 | PO 9 | PO 10 | PO 11 | PO 12 | PSO 1 | PSO 2 |
|-------------|--------------------------------------------------------------------------------|------|------|------|------|------|------|------|------|------|-------|-------|-------|-------|-------|
| CSL700 11.1 | To implement an appropriate machine learning model for the given application.  | 3    | 3    | 3    | 3    | 3    |      |      | 3    | 3    | 3     |       | 3     | 3     |       |
| CSL700 11.2 | To implement ensemble techniques to combine predictions from different models. | 3    | 3    | 3    | 3    | 3    |      |      | 3    | 3    | 3     |       | 3     | 3     |       |
| CSL700 11.3 | To implement the dimensionality reduction techniques.                          | 3    | 3    | 3    | 3    | 3    |      |      | 3    | 3    | 3     |       | 3     | 3     |       |
| Average     |                                                                                | 3    | 3    | 3    | 3    | 3    |      |      | 3    | 3    | 3     |       | 3     | 3     |       |

## Laboratory Assessment

Academic Year: \_\_\_\_\_

Class/Sem: Div:

Batch: \_\_\_\_\_

Student Name: \_\_\_\_\_

Roll No: \_\_\_\_\_

Course Name: \_\_\_\_\_

Course Code: \_\_\_\_\_

### 1. Preparedness and Efforts/ Preparation and Knowledge

3: Well prepared and puts efforts.

2: Not prepared but puts efforts or prepared but doesn't put efforts

1: Neither prepared nor puts efforts

### 2. Presentation of output/ Accuracy and Neatness of Documentation

3: Uses all perfect Instructions / Interrupts/Presented well.

2: Uses some perfect Instructions / Interrupts/ moderate presentation.

1: Doesn't use any of the proper Instruction / Interrupt/ Not presented properly.

### 3. Results/ Participation in Practical Performance

3: Participate and gets proper results

2: Participate but doesn't get proper result or gets result but with the help of faculty in-charge

1: Neither Participate nor gets the results

### 4. Punctuality

3: Get the experiment checked in-time and is always in-time to the lab sessions

2: Some time delays the experiment checking or is late to the lab sessions for few times

1: Most of the time delays experiment checking and / or comes late for lab sessions

### 5. Lab Ethics

3: Follows proper lab ethics by keeping the lab clean and placing things at their right place

2: Sometimes doesn't follow the lab ethics

1: Most of the times makes the lab untidy and keeps things at wrong place

### EVALUATION:

| Performance Indicator/ Expt. No →<br>↓                                   | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | Total |
|--------------------------------------------------------------------------|----|----|----|----|----|----|----|----|----|----|-------|
|                                                                          | CO | CO | CO | CO | CO | CO | CO | CO | CO | CO |       |
| <b>1. Preparedness and Efforts/ Preparation and Knowledge</b>            |    |    |    |    |    |    |    |    |    |    |       |
| <b>2. Presentation of output/ Accuracy and Neatness of Documentation</b> |    |    |    |    |    |    |    |    |    |    |       |
| <b>3. Debugging and results/ Participation in Practical Performance</b>  |    |    |    |    |    |    |    |    |    |    |       |
| <b>4. Punctuality</b>                                                    |    |    |    |    |    |    |    |    |    |    |       |
| <b>5. Lab Ethics</b>                                                     |    |    |    |    |    |    |    |    |    |    |       |
| <b>Total</b>                                                             |    |    |    |    |    |    |    |    |    |    |       |
| <b>Average</b>                                                           |    |    |    |    |    |    |    |    |    |    |       |

Exceed Expectations (3), Meet Expectations (2). Below Expectations (1)

Faculty In-charge

## LIST OF EXPERIMENTS

|     |                                                                       |
|-----|-----------------------------------------------------------------------|
| 1.  | <b>Simple Linear Regression</b>                                       |
| 2.  | <b>Multiple Linear Regression</b>                                     |
| 3.  | <b>Multivariate Linear Regression</b>                                 |
| 4.  | <b>Logistic Regression</b>                                            |
| 5.  | <b>Ensemble Learning: Random Forest</b>                               |
| 6.  | <b>Ensemble Learning: Adaboost</b>                                    |
| 7.  | <b>Support Vector Machine (SVM)</b>                                   |
| 8.  | <b>Graph based clustering: Divisive Clustering</b>                    |
| 9.  | <b>Model based clustering: Expectation Maximization Algorithm</b>     |
| 10. | <b>Dimension Reduction Method: Principal Component Analysis (PCA)</b> |



## PROGRAM NO. 1 SIMPLE LINEAR REGRESSION

**Aim:** To implement simple linear regression in Python.

To understand and implement **Simple Linear Regression** through three approaches:

1. Using parameters  $\beta_0$  and  $\beta_1$  directly.
2. Using **Gradient Descent** for parameter optimization.
3. Using **Python library functions** for regression.

**Theory:**

### Simple Linear Regression:

Simple Linear Regression models the relationship between an independent variable  $X$  and a dependent variable  $Y$  using the equation:

$$Y = \beta_0 + \beta_1 X + \epsilon$$

where:

- $Y$ : Dependent variable (target)
- $X$ : Independent variable (predictor)
- $\beta_0$ : Intercept (value of  $Y$  when  $X=0$ )
- $\beta_1$ : Slope (rate of change of  $Y$  with respect to  $X$ )
- $\epsilon$ : Error term

The aim of Simple Linear Regression is to **find the best-fit line** that minimizes the prediction error.

### Part 1: Using Parameters Directly

In this part, the parameters  $\beta_0$  and  $\beta_1$  are **calculated directly** using the least squares method:

$$\beta_0 = \bar{y} - \beta_1 \bar{x}$$

$$\beta_1 = \frac{\sum_{i=1}^n y_i x_i - \bar{y} n \bar{x}}{(\sum_{i=1}^n x_i^2 - n \bar{x}^2)}$$

$$\bar{x} = \frac{\sum_{i=1}^n x_i}{n} \quad \bar{y} = \frac{\sum_{i=1}^n y_i}{n}$$

This method is efficient for smaller datasets and gives an exact solution by directly solving the equations.

## Part 2: Using Gradient Descent

Gradient Descent is an **iterative optimization technique** used to find the best parameters by minimizing the **Mean Squared Error (MSE)**.

The **cost function (MSE)** is given by:

$$J(\beta_0, \beta_1) = \frac{1}{n} \sum_{i=1}^n (Y_i - (\beta_0 + \beta_1 X_i))^2$$

Using **partial derivatives**, the parameters are updated as:

$$\beta_0 := \beta_0 - \alpha \frac{\partial J}{\partial \beta_0}$$

$$\beta_1 := \beta_1 - \alpha \frac{\partial J}{\partial \beta_1}$$

where:

- $\alpha$  is the learning rate.
- The process continues until the cost function converges to a minimum value.

This method is useful for larger datasets and ensures that the optimal parameters are found iteratively.

### Part 3: Using Python Library Function

Python provides **built-in libraries** like scikit-learn to perform linear regression efficiently. The library function handles the computation of parameters and prediction automatically.

Typical workflow:

1. Fit the model using `LinearRegression().fit(X, Y)`.
2. Use `model.coef_` and `model.intercept_` to extract the parameters  $\beta_1$  and  $\beta_0$  .
3. Predict outcomes for new data using `model.predict(X)`.

This approach abstracts the complexity and is widely used for real-world applications.

### Conclusion:

- **Direct parameter calculation** gives an exact solution but is more suited for smaller datasets.
- **Gradient Descent** is preferred for larger datasets or when iterative improvement is needed.
- **Python libraries** simplify the implementation, making it easier for practical applications.

## PROGRAM NO. 2 MULTIPLE LINEAR REGRESSION

**Aim:** To implement multiple linear regression in Python.

To understand and implement **Multiple Linear Regression** using the following approaches:

1. Using **matrix parameters**  $\beta_0$  and  $\beta$  directly.
2. Using **Gradient Descent** for parameter optimization.
3. Using **Python library functions** for regression.

**Theory:**

### Multiple Linear Regression:

Multiple Linear Regression models the relationship between a dependent variable  $Y$  and multiple independent variables  $X_1, X_2, \dots, X_n$ . The general equation is:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n + \epsilon$$

where:

- $Y$ : Dependent variable (target)
- $X_1, X_2, \dots, X_n$ : Independent variables (predictors)
- $\beta_0$ : Intercept
- $\beta_1, \beta_2, \dots, \beta_n$ : Coefficients for the independent variables
- $\epsilon$ : Error term

The goal is to **find the optimal parameters** that minimize the difference between the predicted and actual values.

### Part 1: Using Matrix Parameters Directly

In matrix form, the equation for Multiple Linear Regression is written as:

$$Y = X\beta + \epsilon$$

where:

- $Y$  is an  $n \times 1$  matrix of observed values
- $X$  is an  $n \times (p+1)$  matrix of predictors (with the first column as 1s for the intercept)
- $B$  is a  $(p+1) \times 1$  matrix of parameters

The **optimal parameters** can be directly computed using the normal equation:

$$\hat{\beta} = (X'X)^{-1}X'Y$$

This method provides an **exact solution**, but it can be computationally expensive for very large datasets.

## Part 2: Using Gradient Descent

Gradient Descent is an **iterative optimization algorithm** used to minimize the **Mean Squared Error (MSE)**:

$$J(\beta_0, \beta_1, \dots, \beta_n) = \frac{1}{n} \sum_{i=1}^n \left( Y_i - \sum_{j=0}^p \beta_j X_{ij} \right)^2$$

The parameters are updated as:

$$\beta_j := \beta_j - \alpha \frac{\partial J}{\partial \beta_j}$$

where  $\alpha$  is the learning rate. The **gradient descent** method is preferred when dealing with large datasets since it iteratively converges to the optimal parameters.

## Part 3: Using Python Library Functions

Python's scikit-learn library simplifies the implementation of Multiple Linear Regression. Key steps include:

1. **Fit the model** using `LinearRegression().fit(X, Y)`.
2. **Extract parameters:** Use `model.coef_` and `model.intercept_` to get the coefficients and intercept.
3. **Make predictions** with `model.predict(X)` on new data.

This approach abstracts complex calculations and is widely used in real-world applications.

### **Conclusion:**

- **Matrix-based computation** provides an exact solution but may struggle with large datasets.
- **Gradient Descent** offers scalability and is effective for larger datasets.
- **Python library functions** are the most convenient for practical implementation, saving time and effort.

## PROGRAM NO. 3 MULTIVARIATE LINEAR REGRESSION

**Aim:** To implement multivariate linear regression in Python.

To implement **Multivariate Linear Regression in Python** and understand the concept using:

1. **One independent variable and multiple dependent variables.**
2. **Multiple independent variables and multiple dependent variables**, utilizing the matrix method for parameter estimation.

**Theory:**

**Multivariate Linear Regression:**

Multivariate Linear Regression refers to regression involving **multiple dependent variables (DV)**. This contrasts with simple multiple regression, where there is only one dependent variable.

The equation for **multivariate linear regression** can be generalized as:

$$Y = X\beta + \epsilon$$

where:

- $Y$ :  $n \times m$  matrix of **dependent variables** (responses), with  $m$  being the number of dependent variables.
- $X$ :  $n \times (p+1)$  matrix of **independent variables** (including a column of 1s for the intercept).
- $\beta$ :  $(p+1) \times m$  matrix of regression coefficients.
- $\epsilon$ :  $n \times m$  matrix representing the error or residuals.

The **optimal parameters**  $\beta$  can be estimated using the **normal equation**:

$$\hat{\beta} = (X^T X)^{-1} X^T Y$$

This method minimizes the sum of squared errors to find the best-fitting line(s).

**Part 1: One Independent Variable and Multiple Dependent Variables (1 Independent Variable, Multiple Dependent Variables)**

**Case Explanation:**

In this case, we have **one independent variable**  $X$  and **multiple dependent variables**  $Y_1, Y_2, \dots, Y_m$ . The goal is to find the regression coefficients for each

dependent variable with respect to the single independent variable.

The equation becomes:

$$Y_j = \beta_0 + \beta_1 X + \epsilon_j \quad \text{for each } j = 1, 2, \dots, m$$

Using the **matrix form**, we compute:

$$\hat{\beta} = (X'X)^{-1}X'Y$$

where  $X$  is a column matrix of the independent variable, and  $Y$  is a matrix of the dependent variables.

## Part 2: Multiple Independent Variables and Multiple Dependent Variables (Multiple Independent Variables, Multiple Dependent Variables)

### Case Explanation:

In this case, we have **multiple independent variables** and **multiple dependent variables**. The goal is to find the optimal coefficients that minimize the residual error for each dependent variable.

The equation becomes:

$$Y_j = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p + \epsilon_j \quad \text{for each } j = 1, 2, \dots, m$$

In **matrix form**, the optimal parameter matrix  $\hat{\beta}$  is again computed as:

$$\hat{\beta} = (X'X)^{-1}X'Y$$

Here, both  $X$  and  $Y$  are matrices with multiple columns representing multiple predictors and responses, respectively.

### Conclusion:

- **Multivariate Linear Regression** allows us to model the relationships between multiple independent and multiple dependent variables.
- Using the **matrix method**, we can efficiently compute the optimal parameters by solving the normal equation.
- **One independent variable with multiple dependent variables** captures simpler relationships, while **multiple independent and dependent variables** provide a more comprehensive model.

## PROGRAM NO. 4 LOGISTIC REGRESSION

**Aim:** To implement logistic regression with **Gradient Descent Approach** for parameter optimization.

### Theory:

**Logistic Regression:** Logistic Regression is a classification algorithm that models the probability of a binary outcome (0 or 1) using the sigmoid function. The predicted probability is constrained between 0 and 1, helping classify data into two categories. Its cost function is the log loss (cross-entropy loss), which ensures accurate probability predictions. Parameters are optimized using Gradient Descent to minimize the loss. Logistic Regression is widely used for tasks like spam detection and disease diagnosis.

Linear Regression and Logistic Regression are both supervised learning algorithms but differ in purpose and output. Linear Regression is used for predicting continuous values, such as house prices, where the dependent variable can take any real number. In contrast, Logistic Regression is used for binary classification problems, such as predicting whether an email is spam or not, where the output is a probability bounded between 0 and 1. Linear Regression models the relationship using a straight line, while Logistic Regression applies a sigmoid function to map predictions to probabilities. The loss function for Linear Regression is Mean Squared Error (MSE), whereas Logistic Regression uses Log Loss (Cross-Entropy) to evaluate classification performance

### Logistic Regression Equation:

Logistic Regression predicts the **probability** of a binary class (0 or 1) using the **sigmoid function**. The predicted probability is given by:

$$P(Y = 1|X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X)}}$$

Here,  $\beta_0$  and  $\beta_1$  are the model parameters to be optimized.

### Cost Function for Logistic Regression:

The **Log Loss** or **Cross-Entropy Loss** is used as the cost function:

$$J(\beta) = -\frac{1}{n} \sum_{i=1}^n [y_i \log(h(x_i)) + (1 - y_i) \log(1 - h(x_i))]$$

where:

- $h(x_i) = \frac{1}{1+e^{-(\beta_0 + \beta_1 x_i)}}$  is the sigmoid output.
- $y_i$  is the actual label for the  $i$ -th sample.

The goal is to **minimize the cost function** by adjusting the parameters  $\beta_0$  and  $\beta_1$ .

### Gradient Descent for Logistic Regression:

The parameters  $\beta_0$  and  $\beta_1$  are updated iteratively using **Gradient Descent**. The update rule for each parameter is:

$$\beta_j := \beta_j - \alpha \frac{\partial J(\beta)}{\partial \beta_j}$$

where:

- $\alpha$  is the learning rate.
- $\frac{\partial J(\beta)}{\partial \beta_j}$  is the gradient of the cost function with respect to parameter  $\beta_j$ .

For logistic regression:

$$\frac{\partial J(\beta)}{\partial \beta_j} = \frac{1}{n} \sum_{i=1}^n (h(x_i) - y_i) x_{ij}$$

### Implementation Outline:

1. **Initialize parameters**  $\beta_0$  and  $\beta_1$  randomly.
2. **Compute predictions** using the sigmoid function.
3. **Calculate the gradient** of the cost function.
4. **Update the parameters** using the gradient descent rule.
5. **Repeat the process** until convergence.

### Conclusion:

- Logistic Regression is a powerful classification technique that uses the sigmoid function to predict probabilities.
- The Gradient Descent algorithm iteratively optimizes parameters to minimize the log loss.

## PROGRAM NO. 5 ENSEMBLE METHOD: RANDOM FOREST

**Aim:** To implement **Ensemble Method: Random Forest** in Python and understand its working and significance.

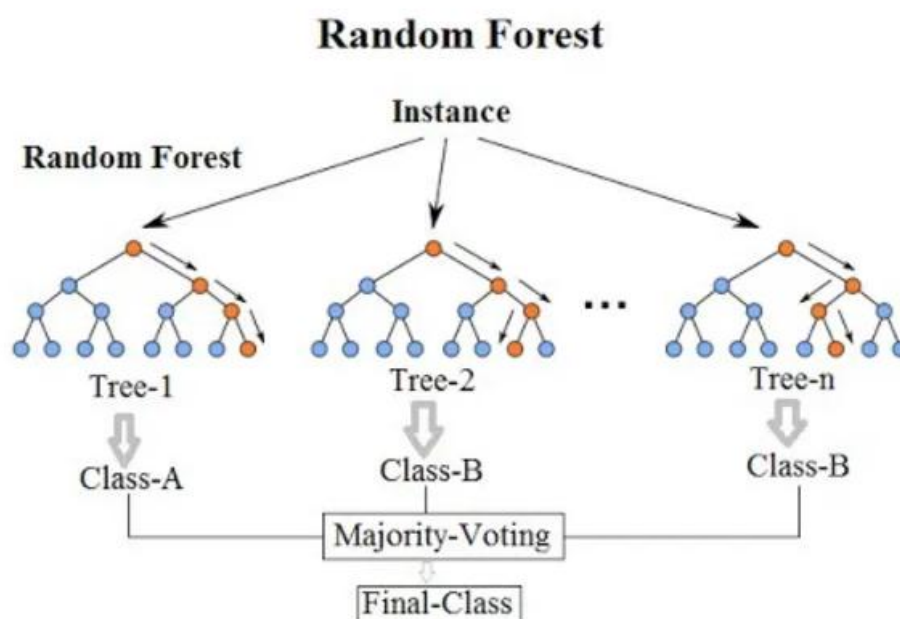
### Theory:

#### Ensemble and Random Forest

Ensemble methods combine multiple models (weak learners) to improve overall performance by reducing variance, bias, or improving predictions. **Random Forest** is a popular ensemble algorithm that constructs multiple **decision trees** during training and outputs the class (for classification) or average prediction (for regression) from all the trees. It introduces **randomness** by selecting random subsets of features and data points for each tree, making the model robust to overfitting and noise.

#### Algorithm – Random Forest:

1. Select random subsets of data and features for each tree (bootstrap sampling).
2. Build a decision tree for each subset without pruning.
3. For classification, use majority voting across trees; for regression, take the average of predictions.
4. Repeat the process for all trees and combine their predictions.



### **Python Implementation:**

1. Import required libraries: `sklearn.ensemble.RandomForestClassifier/Regressor`.
2. Load or generate the dataset and split it into training and testing sets.
3. Train the Random Forest model with the chosen parameters (e.g., number of trees).
4. Use the trained model to predict on the test set and evaluate performance using metrics like accuracy or mean squared error.

### **Conclusion:**

- Random Forest is a powerful and versatile ensemble technique that mitigates overfitting and enhances prediction accuracy through the aggregation of multiple decision trees.
- It is widely used for both classification and regression tasks due to its simplicity, effectiveness, and ability to handle large datasets with high dimensionality.

## **PROGRAM NO. 6 ENSEMBLE METHOD: ADA BOOST**

**Aim:** To implement **Ensemble Method: Ada Boost** in Python and understand its working and significance.

### **Theory:**

#### **Ensemble and AdaBoost – Concept:**

Ensemble methods combine multiple weak learners to improve overall model performance, and AdaBoost (Adaptive Boosting) is one such technique. Unlike Random Forest, which builds trees independently, AdaBoost sequentially builds a series of models, with each new model focusing on the mistakes made by the previous ones. It assigns higher weights to incorrectly classified instances, guiding the next learner to focus more on these errors. The final prediction is made by combining the weighted votes of all models.

#### **Algorithm – AdaBoost:**

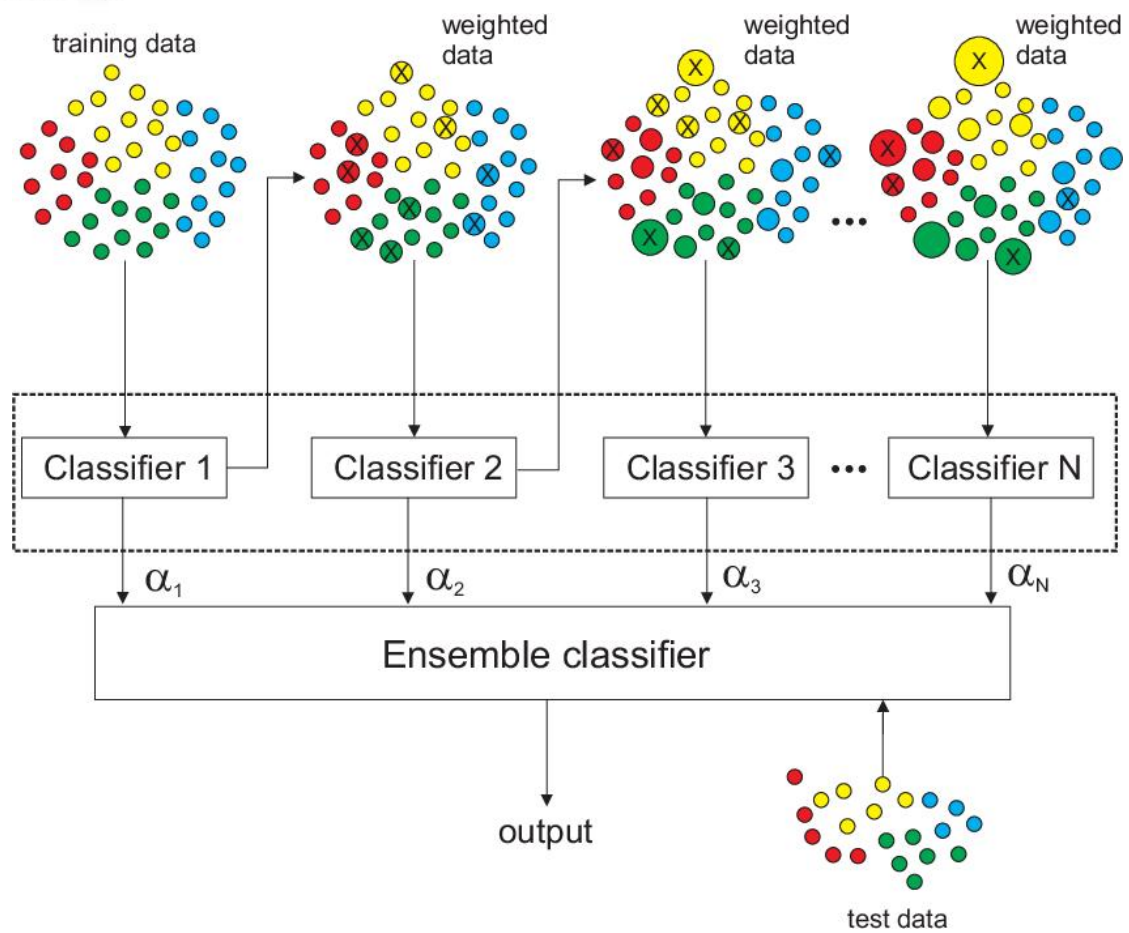
Initialize equal weights for all data points.

Train a weak learner (e.g., a shallow decision tree) on the dataset.

Calculate the weighted error and update the weights of the misclassified points.

Train the next learner with updated weights and repeat for a predefined number of iterations.

Combine predictions using weighted voting for classification or averaging for regression.



### **Python Implementation:**

Import required libraries: `sklearn.ensemble.AdaBoostClassifier/Regressor`.

Load and split the dataset into training and test sets.

Train the AdaBoost model with a chosen number of estimators (weak learners).

Use the trained model to predict on the test set and evaluate performance with accuracy or error metrics. Ensemble and Ada Boost

### **Conclusion:**

- AdaBoost is an effective ensemble method that improves prediction accuracy by focusing on hard-to-classify instances through adaptive weighting.
- It works well with simple base learners and is suitable for both classification and regression tasks, though it may be sensitive to noisy data.

## PROGRAM NO. 7 SUPPORT VECTOR MACHINE (SVM)

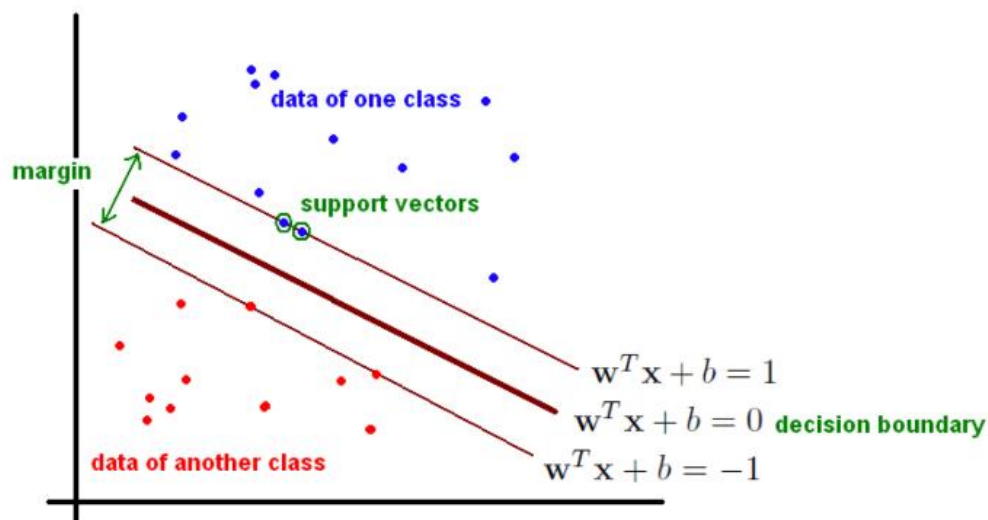
**Aim:** To implement Supervised Support Vector Machine (SVM) in Python and understand its working.

### Theory:

Support Vector Machine (SVM) is a supervised learning algorithm used for both classification and regression tasks. The primary idea is to find the optimal hyperplane that separates data points of different classes with the maximum margin. The margin is the distance between the closest data points (support vectors) of each class to the hyperplane. The decision boundary is defined by the equation  $w^T x + b = 0$ , where  $w$  is the weight vector,  $b$  is the bias, and  $x$  represents the input features. SVM aims to maximize this margin to ensure better generalization.

### Algorithm – SVM:

1. Define the objective to maximize the margin by minimizing  $1/2\|w\|^2$  subject to constraints  $y_i(w^T x_i + b) \geq 1$
2. Solve the optimization problem using **Lagrange multipliers** or **Quadratic Programming**.
3. For non-linearly separable data, apply the **kernel trick** to project data into a higher-dimensional space.
4. Use the trained hyperplane to classify new data points.



### **Python Implementation:**

1. Import required libraries: `sklearn.svm.SVC`.
2. Load or create a dataset and split it into training and test sets.
3. Train the SVM model with a linear or non-linear kernel (e.g., 'rbf', 'poly').
4. Predict the labels for the test set and evaluate the performance using metrics like accuracy or confusion matrix.

### **Conclusion:**

- SVM is a powerful algorithm that finds the optimal decision boundary with the maximum margin, making it effective for high-dimensional datasets.
- It performs well for both linear and non-linear data using kernel methods, though it can be computationally intensive for large datasets.

## PROGRAM NO. 8 DIVISIVE CLUSTERING ALGORITHM based on MST

**Aim:** To implement Divisive Clustering Algorithm based on Minimum Spanning Tree (MST) in Python and understand the graph-based clustering approach.

### Theory:

#### Introduction and Concept:

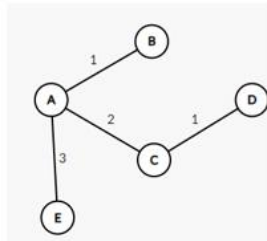
**Divisive Clustering** is a **top-down hierarchical clustering algorithm**, where the entire dataset starts as a single cluster and is iteratively split into smaller clusters until specific criteria are met. **Graph-based clustering** leverages the relationship between data points by representing them as nodes connected by weighted edges, and clusters are formed by splitting the graph. Using a **Minimum Spanning Tree (MST)** for divisive clustering ensures that we partition the data by removing the most significant edges, breaking the dataset along its natural groupings.

#### Algorithm – Divisive Clustering using MST:

1. Represent the dataset as a **graph**, with nodes as data points and edges weighted by the distance (e.g., Euclidean distance) between them.
2. Compute the **Minimum Spanning Tree (MST)** using algorithms like Prim's or Kruskal's to connect all nodes with minimal edge weight.
3. Identify and **remove the largest edge** in the MST, breaking the graph into two clusters.
4. Repeat the process recursively on each new subgraph until the required number of clusters is obtained or stopping criteria are satisfied.

|   | A | B | C | D | E |
|---|---|---|---|---|---|
| A | 0 | 1 | 2 | 2 | 3 |
| B | 1 | 0 | 2 | 4 | 3 |
| C | 2 | 2 | 0 | 1 | 5 |
| D | 2 | 4 | 1 | 0 | 3 |
| E | 3 | 3 | 5 | 3 | 0 |

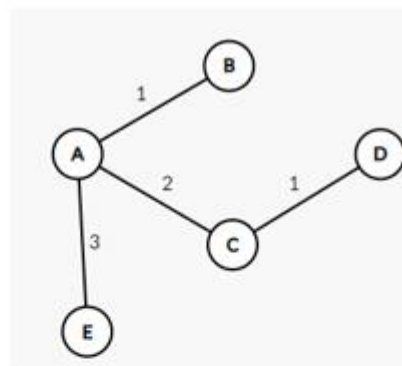
From Adjacency matrix, create Minimum Spanning Tree by Prim's or Kruskal's Algorithm



| Edge | Cost |
|------|------|
| A-B  | 1    |
| C-D  | 1    |
| A-C  | 2    |
| A-D  | 2    |
| B-C  | 2    |
| A-E  | 3    |
| B-E  | 3    |
| D-E  | 3    |
| B-D  | 4    |
| C-E  | 5    |

| Edge | Cost |
|------|------|
| A-B  | 1    |
| C-D  | 1    |
| A-C  | 2    |
| A-D  | 2    |
| B-C  | 2    |
| A-E  | 3    |
| B-E  | 3    |
| D-E  | 3    |
| B-D  | 4    |
| C-E  | 5    |

x  
x  
x  
x  
x  
x



Largest edge is between **A & E**.

Cutting this edge results into two clusters **{E}** and **{A, B, C, D}**.

Next, remove the edge between **A & C**.

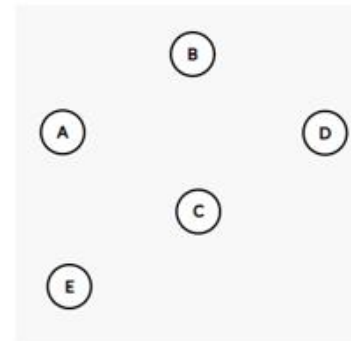
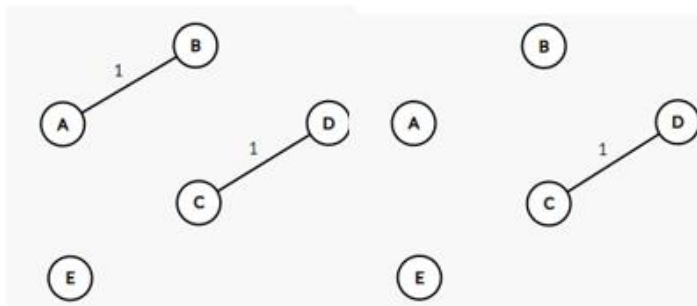
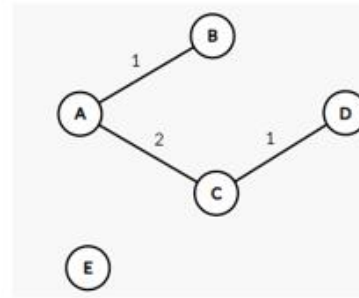
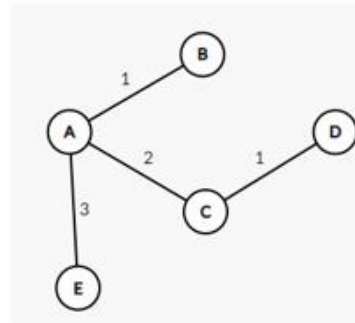
This splits creates three clusters **{A, B}** ,**{C, D}** and **{E}**.

Next break **A & B**

This splits creates four clusters **{A}** , **{B}** ,**{C, D}** and **{E}**.

Next break **C & D**

This splits creates five clusters **{A}** , **{B}** ,**{C}**,**{D}** and **{E}**.



### Python Implementation:

1. Import necessary libraries: networkx (for graph and MST operations) and scipy (for distance computation).
2. Create a graph using the dataset and compute the MST.
3. Identify the edge with the largest weight and remove it to split the graph.
4. Continue splitting iteratively to form clusters, and visualize or evaluate the clusters formed.

### Conclusion:

- Divisive clustering using **MST** provides an effective way to cluster data by capturing the natural structure of the graph.
- This approach works well when data points form clusters with varying densities and shapes. Although **computationally intensive** for large datasets, MST-based clustering ensures meaningful partitions that are difficult to achieve with traditional distance-based methods.

## PROGRAM NO. 9 EXPECTATION MAXIMIZATION (EM) CLUSTERING ALGORITHM

**Aim:** To implement Model-Based EM (Expectation-Maximization) Clustering Algorithm in Python and understand its theoretical foundation and working.

### Theory:

The **Expectation-Maximization (EM) algorithm** is a **model-based clustering approach** that assumes the data is generated from a mixture of distributions (usually Gaussian). The goal is to estimate the parameters of each distribution and assign data points to clusters. It iteratively applies two steps:

1. **Expectation Step (E-Step):** Estimate the probability (responsibility) that each data point belongs to each cluster based on the current parameters.
2. **Maximization Step (M-Step):** Update the parameters (mean, covariance, and mixing coefficients) to maximize the likelihood of the observed data given the estimated responsibilities.

The EM algorithm continues until convergence, ensuring that the **log-likelihood** of the data improves with each iteration.

### Expectation-Maximization (EM) Steps for Gaussian Mixture Model:

EM to estimate the two means. EM algorithm first initializes the hypothesis to  $h = (\mu_1, \mu_1)$ , where  $\mu_1$  and  $\mu_2$  are arbitrary initial values. It then iteratively re-estimates  $h$  by repeating the following two steps until the procedure converges to a stationary value for  $h$ .

#### Step 1:

Calculate the expected value  $E[z_{ij}]$  of each hidden variable  $z_{ij}$ , assuming the current hypothesis  $h = (\mu_1, \mu_2)$  holds.

#### Step 2:

Calculate a new maximum likelihood hypothesis  $h' = (\mu'_1, \mu'_2)$ , assuming the value taken on by each hidden variable  $z_{ij}$  is its expected value  $E[z_{ij}]$  calculated in Step 1. Then replace the hypothesis  $h = (\mu_1, \mu_2)$  with the new hypothesis  $h' = (\mu'_1, \mu'_2)$  and iterate.

Step 1 must calculate the expected value of each  $z_{ij}$ .

This  $E[z_{ij}]$  is just the probability that instance  $x_i$  was generated by the  $j$ -th Normal distribution.

$$E[z_{ij}] = \frac{p(x = x_i | \mu = \mu_j)}{\sum_{n=1}^2 p(x = x_i | \mu = \mu_n)}$$

Since  $p(x = x_i | \mu_j)$  follows a Gaussian distribution:

$$E[z_{ij}] = \frac{e^{-\frac{1}{2\sigma^2}(x_i - \mu_j)^2}}{\sum_{n=1}^2 e^{-\frac{1}{2\sigma^2}(x_i - \mu_n)^2}}$$

Thus, the first step is implemented by substituting the current values  $(\mu_1, \mu_2)$  and the observed  $x_i$  into the above expression.

In Step 2, the  $E[z_{ij}]$  values calculated during Step 1 are used to derive a new maximum likelihood hypothesis  $h' = (\mu'_1, \mu'_2)$ . As discussed later, the new maximum likelihood hypothesis in this case is given by:

$$\mu'_j = \frac{\sum_{i=1}^n E[z_{ij}] \cdot x_i}{\sum_{i=1}^n E[z_{ij}]}$$

### Python Implementation:

1. Import necessary libraries: `sklearn.mixture.GaussianMixture`.
2. Load the dataset and initialize the Gaussian Mixture Model (GMM) for the desired number of clusters.
3. Fit the model using `GMM.fit()` and predict the cluster assignments.
4. Visualize the clusters and evaluate performance metrics like BIC (Bayesian Information Criterion) or AIC (Akaike Information Criterion).

### Conclusion:

- The EM algorithm for clustering provides a **probabilistic approach** that can handle complex data distributions by fitting multiple Gaussian components.
- Unlike distance-based clustering, EM clustering assigns data points based on probabilities, making it robust to noise and overlapping clusters.

## PROGRAM NO. 10 PRINCIPAL COMPONENT ANALYSIS (PCA)

**Aim:** To implement Principal Component Analysis (PCA) in Python for dimensionality reduction.

### Theory:

Principal Component Analysis (PCA) is a dimensionality reduction technique that transforms data from its original axis to a new axis where the variance is maximized. By projecting the data onto these new axes (called principal components), it becomes possible to retain the most informative aspects of the data while reducing noise and redundancy. PCA helps in visualizing high-dimensional data and speeds up computation by reducing the number of features without significant information loss.

### Algorithm Steps:

#### Standardization:

Scale the data so that each feature has a mean of 0 and a standard deviation of 1. This step ensures that all features contribute equally, preventing biases caused by differences in scales.

#### Compute Covariance Matrix:

Calculate the covariance matrix of the standardized data to understand the relationships between variables (features)

$$C = \frac{1}{n-1} X^T X$$

Where X is the standardized data matrix and n is the number of samples.

#### Calculate Eigenvalues and Eigenvectors:

Compute the eigenvalues and eigenvectors of the covariance matrix.

**Eigenvectors** represent the new directions (axes).

**Eigenvalues** correspond to the amount of variance along each eigenvector.

#### Sort Eigenvalues and Eigenvectors:

Rank the eigenvalues in descending order and arrange the corresponding eigenvectors accordingly. The eigenvectors with the highest eigenvalues represent the most important principal components.

**Select Principal Components:**

Choose the top  $k$  eigenvectors based on the desired level of variance to retain in the transformed data. This determines the reduced number of dimensions.

**Transform Data:**

Project the original data onto the selected principal components to obtain the lower-dimensional representation.

$$X_{new} = X \cdot W$$

Where  $W$  is the matrix of selected eigenvectors (principal components).

**Conclusion:**

- PCA reduces the dimensionality of data by identifying and projecting it onto directions where variance is maximized.
- This technique is widely used for feature extraction, visualization of high-dimensional data, and speeding up machine learning models.